

Flavour Effects in Early-Universe Leptogenesis

A Theoretical Study of CP Violation Impact

Mariam Harbi

November 5, 2025

Abstract

We present a theoretical study of the evolution of lepton asymmetry in the early Universe, focusing on the impact of CP violation and flavour-dependent effects within the type-I Seesaw framework. By incorporating three heavy Majorana neutrinos (N_i) and solving the corresponding Boltzmann equations numerically, we explore how flavour-dependent CP asymmetries (ε_i^α) generate a net $(B - L)$ asymmetry. The analysis accounts for decays, inverse decays, $\Delta L = 1$ scatterings, and $\Delta L = 2$ processes, with the dynamical evolution expressed in terms of the dimensionless variable $z = M_1/T$. We demonstrate how the interplay of CP-violating parameters and flavour effects leads to a non-zero lepton asymmetry, which can subsequently be converted to baryon asymmetry through sphaleron transitions. The final asymmetry is compared to the observed baryon asymmetry of the Universe ($Y_B \sim 8.7 \times 10^{-11}$), highlighting the quantitative relevance of flavour effects in early-Universe leptogenesis.

1 Introduction

The observed matter-antimatter asymmetry in the Universe, quantified by the baryon asymmetry

$$Y_B = \frac{n_B - n_{\bar{B}}}{s}, \quad (1.1)$$

remains one of the central puzzles of modern cosmology. The Universe, as inferred from the Cosmic Microwave Background (CMB), began in a nearly symmetric state, yet CP-violating processes during the early stages of cosmic evolution generated a net baryon number.

Leptogenesis provides a compelling mechanism to account for this asymmetry. In this framework, the out-of-equilibrium decays of heavy Majorana neutrinos (N_i) produce a lepton asymmetry ($Y_{\Delta L}$), which is partially converted into baryon asymmetry through non-perturbative sphaleron processes. While early studies considered lepton flavours collectively, recent developments have emphasized the significance of flavour-dependent effects, as distinct flavours (e, μ, τ) experience different interactions with the thermal bath.

The goal of this project is to investigate how CP violation and flavour effects influence the generation and evolution of lepton asymmetries, using a numerical solution of the Boltzmann equations for heavy neutrinos and flavour-dependent lepton densities.

2 Theoretical Framework

2.1 Seesaw Type-I Model

We adopt the type-I Seesaw framework by extending the Standard Model with three heavy Majorana neutrinos ($N_i, i = 1, 2, 3$). The light neutrino mass matrix arises via the standard Seesaw relation:

$$m_\nu \approx -m_D^T M^{-1} m_D, \quad (2.1)$$

where m_D is the Dirac mass matrix and $M = \text{diag}(M_1, M_2, M_3)$ is the Majorana mass matrix of heavy neutrinos. The Yukawa couplings (Y_ν) connect N_i to the Standard Model lepton doublets (L_α) and the Higgs (H), and play a key role in generating CP violation.

Parameters of interest:

- Heavy neutrino masses (M_i)

- Yukawa couplings (Y_ν)
- Mixing angles (θ_{ij}) and CP-violating phases

2.2 Boltzmann Equations

The evolution of the heavy neutrino abundances (Y_{N_i}) and lepton flavour asymmetries (Y_{Δ_α}) is described by the coupled Boltzmann equations:

$$\frac{dY_{N_i}}{dz} = -D_i(z) \left(Y_{N_i}(z) - Y_{N_i}^{eq}(z) \right), \quad (2.2)$$

$$\frac{dY_{\Delta_\alpha}}{dz} = - \sum_i \varepsilon_i^\alpha D_i(z) \left(Y_{N_i}(z) - Y_{N_i}^{eq}(z) \right) - W_\alpha(z) Y_{\Delta_\alpha}(z), \quad (2.3)$$

where $z = M_1/T$, $D_i(z)$ denote decay terms, $W_\alpha(z)$ denote washout processes, and ε_i^α are the flavour-dependent CP asymmetry parameters. The equilibrium yields are given by:

$$Y_{N_i}^{eq}(z) = \frac{45g_N}{4\pi^4 g_s^*} r_i^2 z^2 K_2(r_i z), \quad (2.4)$$

with $r_i = M_i/M_1$ and K_2 the modified Bessel function of the second kind.

2.3 CP Violation and Flavour Effects

The CP asymmetry parameters ε_i^α quantify the difference in decay rates of N_i to leptons versus antileptons:

$$\varepsilon_i^\alpha = \frac{\Gamma(N_i \rightarrow L_\alpha H) - \Gamma(N_i \rightarrow \bar{L}_\alpha H^\dagger)}{\sum_\beta [\Gamma(N_i \rightarrow L_\beta H) + \Gamma(N_i \rightarrow \bar{L}_\beta H^\dagger)]}. \quad (2.5)$$

Flavour effects are crucial because interactions with the thermal bath differentiate the evolution of electron, muon, and tau asymmetries. Ignoring flavour can significantly misestimate the final baryon asymmetry.

3 Numerical Implementation

3.1 Initial Conditions

We set the initial conditions at $z_{\text{start}} \ll 1$, where the heavy neutrinos are in equilibrium:

$$Y_{N_i}(z_{\text{start}}) = Y_{N_i}^{eq}(z_{\text{start}}), \quad Y_{\Delta_\alpha}(z_{\text{start}}) = 0. \quad (3.1)$$

3.2 Parameters and Functions

Numerical values for the parameters are chosen consistent with a hierarchical scenario:

- $M_1 = 1 \text{ TeV}, M_2 = 2.5 \text{ TeV}, M_3 = 18 \text{ TeV}$
- CP asymmetries ε_i^α taken from a representative Yukawa texture
- Decay functions: $D_i(z) = \Gamma_i / [H(T = M_i/z)z]$
- Washout terms $W_\alpha(z)$ include $\Delta L = 1$ and $\Delta L = 2$ scatterings
- Cosmological functions: $H(T) = \sqrt{\frac{8\pi^3 g_*}{90}} \frac{T^2}{M_{\text{Pl}}}, s(T) = \frac{2\pi^2}{45} g_*^* T^3$

3.3 Numerical Solver

The Boltzmann equations are solved using adaptive Runge–Kutta methods (SciPy's `solve_ivp`) across the z -range. This yields the evolution of $Y_{N_i}(z)$ and $Y_{\Delta_\alpha}(z)$, which are summed to compute Y_{B-L} .

4 Results and Discussion

- Equilibrium and out-of-equilibrium evolution: $Y_{N_i}(z)$ depart from equilibrium at $z \sim 1$, triggering lepton asymmetry generation.
- Flavour-dependent asymmetries: $Y_{\Delta_e}, Y_{\Delta_\mu}, Y_{\Delta_\tau}$ evolve differently due to both ε_i^α and flavour-dependent washouts.
- Final baryon asymmetry: The net $(B - L)$ asymmetry converts to baryon asymmetry via sphaleron processes:

$$Y_B = c_s \sum_{\alpha} Y_{\Delta_\alpha}, \quad c_s = -\frac{28}{79}. \quad (4.1)$$

Numerical results yield an asymmetry of order $Y_B \sim 10^{-10}$, consistent with observational data.

- Impact of CP violation and flavours: The final baryon asymmetry is highly sensitive to both the structure of ε_i^α and the relative strength of washout terms for each flavour.

5 Conclusion

This study demonstrates the critical role of flavour effects and CP violation in early-Universe leptogenesis. Using a type-I Seesaw framework with heavy Majorana neutrinos, we showed numerically that the evolution of flavour-dependent lepton asymmetries, governed by Boltzmann equations, can produce a baryon asymmetry compatible with observations.

Future improvements may include:

- Incorporation of finite-temperature corrections to decay rates
- More precise treatment of $\Delta L = 1, 2$ scattering cross-sections
- Inclusion of fully flavour-covariant Boltzmann equations
- Exploration of different neutrino mass hierarchies and Yukawa textures

Appendix: Full Python Code

```

1  # Flavoured Leptogenesis: symbolic + numeric framework
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from scipy import special, integrate
5  import sympy as sp
6  sp.init_printing(use_unicode=True)
7  # MODE: 'symbolic'          'numeric'
8  # MODE = 'numeric'
9
10 Mpl_red = 2.435e18          # reduced Planck mass [GeV]
11 gstar = 106.75
12 gstar_s = 106.75
13 vEW = 246.0                # Higgs VEV [GeV] (standard)
14 gN = 2.0                   # degrees of freedom for Majorana N
15 #
16                               : z, M_i, Yukawa entries, epsilon_flavour
17 z = sp.symbols('z', positive=True)          # z = M1/T (dimensionless)
18 M1_s, M2_s, M3_s = sp.symbols('M1 M2 M3', positive=True)
19 # Yukawa matrix symbols Y_{alpha i} (alpha = e,mu,tau indexed 0,1,2)
20 Y = sp.Matrix([[sp.symbols(f'Y_e{i+1}')] for i in range(3)],
21               [sp.symbols(f'Y_mu{i+1}')] for i in range(3)],
22               [sp.symbols(f'Y_tau{i+1}')] for i in range(3)])
23 # flavoured CP asymmetries eps_{i,alpha}
24 eps_sym = sp.Matrix([[sp.symbols(f'eps_{i+1}_{a}')] for a in range(3)]
25                     for i in range(3)])
26
27 # helper symbolic functions (leave Bessel as symbolic functions)
28 K1 = sp.Function('K1') # K1(x)
29 K2 = sp.Function('K2') # K2(x)
30
31 # equilibrium yield (general) for N_i (symbolic expression)
32 # Y_N^eq(z; M_i) = (45 g_N / (4 pi^4 g*_s)) * ( (M_i/M1)^2 * z^2 * K2((M_i
33               /M1)*z) )
34 pref_Yeq = (45 * gN) / (4 * sp.pi**4 * gstar_s)
35 r1, r2, r3 = sp.symbols('r1 r2 r3', positive=True) # r_i = M_i / M1
36 Yeq_sym_i = lambda ri: pref_Yeq * ri**2 * z**2 * sp.Function('K2')(ri*z
37 )
38
39 # Decay parameter D_i (symbolic form)          using argument x_i = r_i * z
40 # D_i(z) = (Gamma_i / H(T=M_i/z)) * z * K1(x_i)/K2(x_i)
41 # We'll leave Gamma_i symbolic as Gamma_i (depends on Yukawas)
42 Gamma1_s, Gamma2_s, Gamma3_s = sp.symbols('Gamma1 Gamma2 Gamma3',
43               positive=True)

```

```

39 H_sym = lambda T: sp.sqrt(8*sp.pi**3 * gstar / 90) * T**2 / Mpl_red #
    symbolic form (T remains symbol)
40 # Represent H at T = Mi/z as H_i(z) ~ sqrt(...) * (Mi/z)^2 / Mpl -> we'
    ll show pattern
41 # For clarity keep D_i symbolic:
42 ri = sp.symbols('ri', positive=True)
43 D_sym = sp.Function('D') # represent D_i symbolically
44
45 # Flavoured Boltzmann equations (symbolic form)
46 # For each i (heavy neutrino): dY_Ni/dz = - D_i(z) * (Y_Ni - Y_Ni^eq)
47 # For each flavour alpha: dY_Delta_alpha/dz = - sum_i eps_{i,alpha} D_i
    (Y_Ni - Y_Ni^eq) - W_alpha(z) Y_Delta_alpha
48 Y_N1_sym, Y_N2_sym, Y_N3_sym = sp.symbols('Y_N1 Y_N2 Y_N3')
49 YDL1, YDL2, YDL3 = sp.symbols('Y_DL_e Y_DL_mu Y_DL_tau')
50
51 Y_N1 = sp.Function('Y_N1')(z)
52 Y_N2 = sp.Function('Y_N2')(z)
53 Y_N3 = sp.Function('Y_N3')(z)
54
55 dY_N1 = -sp.Function('D1')(z) * (Y_N1 - Yeq_sym_i(r1))
56 dY_N2 = -sp.Function('D2')(z) * (Y_N2 - Yeq_sym_i(r2))
57 dY_N3 = -sp.Function('D3')(z) * (Y_N3 - Yeq_sym_i(r3))
58
59 # flavoured dYDL (symbolic, show summation compactly)
60 # dY_DL_alpha = - sum_i eps_{i,alpha} D_i (Y_Ni - Y_Ni^eq) - W_alpha(z)
    * Y_DL_alpha
61 dYDL_alpha = lambda alpha: -sum(
62     eps_sym[i, alpha] * sp.Function(f'D{i+1}')(z) *
63     (sp.Function(f'Y_N{i+1}')(z) - Yeq_sym_i(sp.symbols(f'r{i+1}'))))
64     for i in range(3)
65 ) - sp.Function(f'W_{["e","mu","tau"][alpha]}')(z) * sp.Function(f'
    Y_DL_{["e","mu","tau"][alpha]}')(z)
66 # If MODE == 'symbolic' -> print and return clean symbolic forms
67 if MODE == 'symbolic':
68     print("\n=== SYMBOLIC REPRESENTATION (summary) ===\n")
69     print("Equilibrium yield (symbolic) for Ni:")
70     print("Y_N^eq_i(z) = pref * r_i^2 * z^2 * K2(r_i z)   where pref =
        45 g_N / (4 pi^4 g*_s)")
71     sp.pprint(pref_Yeq)
72     print("\nExample: Y_N^eq (for r_i) =")
73     sp.pprint(Yeq_sym_i(ri))
74     print("\nSymbolic Boltzmann (heavy neutrinos):")
75     print("dY_N1/dz = - D1(z) * (Y_N1(z) - Y_N1^eq(z))")
76     sp.pprint(dY_N1)
77     print("\nSymbolic flavoured Boltzmann (generic alpha):")
78     sp.pprint(dYDL_alpha(0)) # print for alpha=0 (electron flavour)

```

```

79     print("\nNote: D_i(z) left as symbolic function (depends on Gamma_i
        and H(T=Mi/z)).")
80     print("You can substitute explicit Gamma_i expressions (e.g.,
        Gamma_i = (Y^ Y )_{ii} Mi/(8 pi))")
81     print("\nSphaleron conversion (symbolic): Y_B = c_s * sum_alpha
        Y_DL_alpha")
82     c_s = sp.Rational(-28,79)
83     sp.pprint(c_s)
84
85     print("\n\n=== END SYMBOLIC SUMMARY ===\n")
86     # End of symbolic mode
87     raise SystemExit("Symbolic mode: finished printing symbolic
        framework. Set MODE='numeric' to run numeric solver.")
88 # 4) NUMERIC MODE: substitute values and solve
89 # If here, MODE=='numeric'
90 # (Note: careful with epsilon magnitudes      we print a warning if any
        |eps|>1)
91 # Default numeric parameters (you can edit these benchmark values)
92 M1 = 1e10
93 M2 = 5e11
94 M3 = 1e13
95 M_list = np.array([M1, M2, M3])
96 r = M_list / M1
97
98 # Yukawa (example approximate values      editable)
99 Yukawas = np.array([1e-3, 5e-4, 2e-4]) # these are diagonal effective
        couplings used for Gamma estimate
100
101 # Decay widths (simple Yukawa formula Gamma = y^2 M /(8 pi))
102 Gamma = (Yukawas**2 * M_list) / (8.0 * np.pi)
103
104 # CP asymmetries (from text examples) -- CAUTION: some paper values can
        be >1; check physicality.
105 eps = np.array([-0.066, -1.012, 0.0005]) # default from provided
        benchmarks (user provided)
106 if np.any(np.abs(eps) > 1.0):
107     print("WARNING: Some |eps| > 1. Ensure these are per-decay
        asymmetries or rescale appropriately for physicality.")
108
109 # H(T) and s(T) numeric functions
110 def H_of_T_num(T, gstar_local=gstar, Mpl=Mpl_red):
111     return np.sqrt(8 * np.pi**3 * gstar_local / 90.0) * T**2 / Mpl
112
113 def s_of_T_num(T, gstar_s_local=gstar_s):
114     return (2.0 * np.pi**2 / 45.0) * gstar_s_local * T**3
115
116 # Equilibrium yield numeric

```

```

117 def Y_N_eq_num(z, Mi, Mref=M1):
118     T = Mref / z
119     x = Mi / T # = (Mi/Mref) * z
120     K2 = special.kv(2, x)
121     n_eq = gN / (2.0 * np.pi**2) * Mi**2 * T * K2
122     s = s_of_T_num(T)
123     return n_eq / s
124
125 # D_i(z) numeric (corrected: use H evaluated at T = Mi/z and Bessel
    args = x_i = (Mi/M1)*z)
126 def H_at_z(z, Mi):
127     T = Mi / z
128     return H_of_T_num(T)
129
130 def D_i_num(z, i):
131     xi = r[i] * z
132     K1 = special.kv(1, xi)
133     K2 = special.kv(2, xi)
134     # Avoid division by zero for tiny xi
135     if K2 == 0:
136         return 0.0
137     return (Gamma[i] / H_at_z(z, M_list[i])) * z * K1 / K2
138
139 # Simple washout terms
140 # W_ID (inverse-decay) standard approximations
141 def W_ID_num(z, i):
142     # using often-used form W_ID = 0.5 * D_i(z) * Y_N_eq(z) / Y_l_eq (
        we approximate Y_l_eq ~ constant ~ 3/4 * 45/(2*pi^4*gstar_s)
        ???)
143     # For simplicity, use W_ID = 0.5 * D_i * (Y_N_eq / Y_ref) with
        Y_ref = 1e-3 typical scale (user can refine)
144     Yref = 1e-3
145     return 0.5 * D_i_num(z, i) * (Y_N_eq_num(z, M_list[i]) / Yref)
146
147 # L =2 washout rough approximation (uses sum m_nu^2)
148 mnu_vals = np.array([0.0, 8.6e-3, 0.05]) # sample light neutrino
        masses [eV] (editable)
149 mbar2 = np.sum(mnu_vals**2)
150 # conversion eV^2 -> GeV^2 : 1 eV = 1e-9 GeV
151 mbar2_GeV2 = mbar2 * (1e-9)**2
152
153 def W_DeltaL2_num(z, i):
154     Mi = M_list[i]
155     # approximate pre-factor from earlier derivation
156     return 4.49e-3 * (Mi * Mpl_red) / (gstar**1.5 * vEW**4) * (
        mbar2_GeV2) / (z**2)
157

```

```

158 # Boltzmann RHS numeric (3 Ni + 3 flavours tracked as vector of length
    6: Y_N1,Y_N2,Y_N3, Y_DL_e,mu,tau)
159 def boltzmann_numeric(z, y):
160     YN = y[0:3]
161     YDL = y[3:6]
162     dY = np.zeros(6)
163     # Heavy neutrino equations
164     for i in range(3):
165         Di = D_i_num(z, i)
166         Yeqi = Y_N_eq_num(z, M_list[i])
167         dY[i] = - Di * (YN[i] - Yeqi)
168     # flavour equations (flavour-dependent washout included: inverse
        decay + DeltaL2)
169     for alpha in range(3):
170         prod = 0.0
171         wash = 0.0
172         for i in range(3):
173             Di = D_i_num(z, i)
174             Yeqi = Y_N_eq_num(z, M_list[i])
175             prod += eps[i] * Di * (YN[i] - Yeqi) # NOTE: here eps is
                flavour-averaged; extend eps->matrix for true flavoured
176             wash += W_ID_num(z, i) + W_DeltaL2_num(z, i)
177         dY[3+alpha] = - prod - wash * YDL[alpha]
178     return dY
179
180 # Initial conditions (thermal abundance for N's and zero lepton
    asymmetry)
181 z0 = 0.1
182 zmax = 200.0
183 z_vals_num = np.logspace(np.log10(z0), np.log10(zmax), 300)
184
185 Y0 = np.zeros(6)
186 # start N densities at equilibrium at z0
187 for i in range(3):
188     Y0[i] = Y_N_eq_num(z0, M_list[i])
189 Y0[3:6] = 0.0
190
191 # Solve with stiff solver BDF for speed
192 sol = integrate.solve_ivp(boltzmann_numeric, [z_vals_num[0], z_vals_num
    [-1]], Y0,
193                             t_eval=z_vals_num, method='BDF', rtol=1e-6,
                                atol=1e-12, max_step=1.0)
194
195 if not sol.success:
196     print("Warning: ODE solver failed:", sol.message)
197
198 # Extract results

```

```

199 Y_Ns = sol.y[0:3,:]
200 Y_DL_flavours = sol.y[3:6,:]
201 Y_DL_total = np.sum(Y_DL_flavours, axis=0)
202
203 # Sphaleron conversion:  $Y_B = c_s * Y_{\{B-L\}}$  (approx  $c_s = -28/79$ )
204 c_s = -28.0/79.0
205 Y_B = c_s * Y_DL_total
206
207 # Final values
208 Y_DL_final = Y_DL_total[-1]
209 Y_B_final = Y_B[-1]
210
211 print("\n=== Numeric run summary ===")
212 print(f"Final lepton asymmetry (sum flavours) Y_ L = {Y_DL_final:.3e}"
213       )
214 print(f"Converted baryon asymmetry Y_B = {Y_B_final:.3e}")
215 print("Note: check magnitude & sign; physical observed Y_B ~ +8.7e-11 (
216       positive)")
217
218 # Plotting
219 plt.figure(figsize=(8,6))
220 for i in range(3):
221     plt.loglog(z_vals_num, Y_Ns[i,:], label=f'Y_N{i+1}')
222 plt.loglog(z_vals_num, np.abs(Y_DL_total), linestyle='--', label='|
223     Y_ L | (sum flavours)')
224 plt.xlabel('z = M1/T')
225 plt.ylabel('Yield')
226 plt.title('Numeric evolution (example benchmarks)')
227 plt.legend()
228 plt.grid(which='both', ls=':', lw=0.6)
229 plt.show()

```